

中国海洋大学计算机科学与技术学院

实验二 汇编实验

姓名：吴虹霖 年级：2024 级 专业： 计算机科学与技术

科目：计算机系统基础 题目： 实验二

实验时间: 2025 年月 13 日 实验教师:蒋永国

实验一、IBM-PC 寻址方式及 DEBUG 的使用

一、实验目的：

- 1.熟悉 DEBUG 的功能，运行并掌握其常用命令。
- 2.使用 DEBUG，通过数据传送指令观察 IBM-PC 机各种寻址方式的区别。

二、实验任务及要求：

实验内容：

1. DEBUG 的加载及其常用命令（A、U、R、D、E、T、P、G、Q）的使用情况。（注意：微机进入 DEBUG 状态下之后,一切立即数和地址数据均被默认为十六进制数,在输入时数的后面不加后缀“H”。）

2.将数据段中的两个数求和，结果保存在数据段中。

源程序、机器语言程序如下：

CS: 0100 A1 00 00 MOV AX, [0000]

CS: 0103 03 06 02 00 ADD AX, [0002]

CS: 0107 A3 04 00 MOV [0004], AX

CS: 010A CB RETF

数据段内容如下：

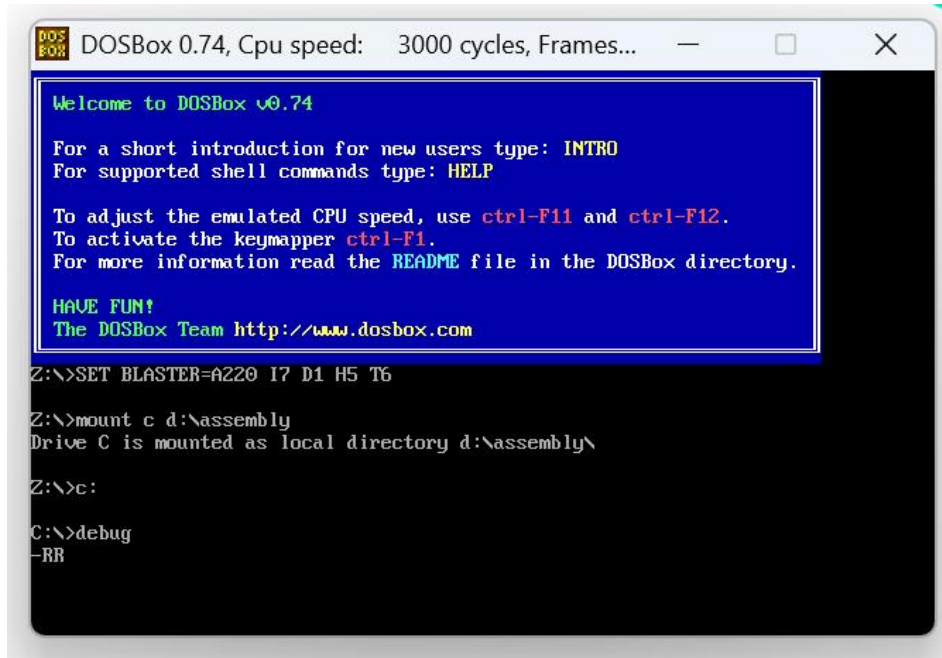
DS: 0000 2301 2500 ; 两个已知数

3.在 Debug 中编制简单的汇编程序片断，并调试运行，得出结果。

三、实验过程及结果：

1.在 DOS 或 Windows 的命令行窗口执行命令 Debug.exe, 进入 Debug 程序环境, 熟悉 Debug 各个命令的用法。

将文件储存在 D 盘 Assembly 文件夹中, 并将本地文件夹挂载为 DOSBox 的 C 盘。随后切换到 c 盘输入 debug, 进入程序环境。



```
DOSBox 0.74, Cpu speed: 3000 cycles, Frames...  
Welcome to DOSBox v0.74  
For a short introduction for new users type: INTRO  
For supported shell commands type: HELP  
  
To adjust the emulated CPU speed, use ctrl-F11 and ctrl-F12.  
To activate the keymapper ctrl-F1.  
For more information read the README file in the DOSBox directory.  
  
HAVE FUN!  
The DOSBox Team http://www.dosbox.com  
  
Z:\>SET BLASTER=A220 I7 D1 H5 T6  
  
Z:\>mount c d:\assembly  
Drive C is mounted as local directory d:\assembly\  
  
Z:\>c:  
  
C:\>debug  
-RR
```

2.实验步骤

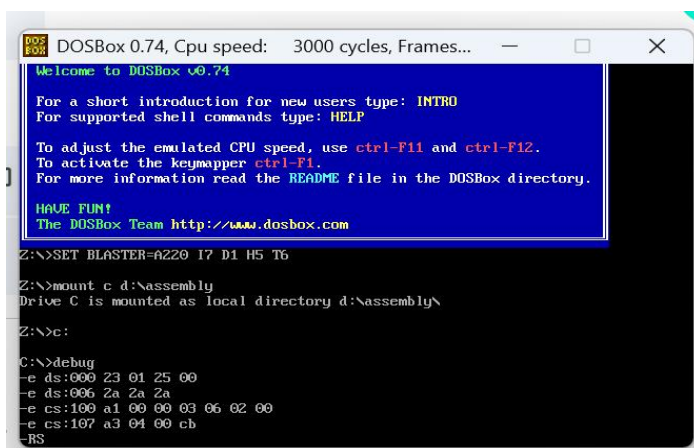
(1) 用 E 命令键入上述机器语言程序和数据段的初值。

—E DS: 000 2301 2500

—E DS: 006 2A 2A 2A (2A 为*, 是为便于查看数据段内容而设置)

—E CS: 100 A1 00 00 03 06 02 00

—E CS: 107 A3 04 00 CB



```
DOSBox 0.74, Cpu speed: 3000 cycles, Frames...  
Welcome to DOSBox v0.74  
For a short introduction for new users type: INTRO  
For supported shell commands type: HELP  
  
To adjust the emulated CPU speed, use ctrl-F11 and ctrl-F12.  
To activate the keymapper ctrl-F1.  
For more information read the README file in the DOSBox directory.  
  
HAVE FUN!  
The DOSBox Team http://www.dosbox.com  
  
Z:\>SET BLASTER=A220 I7 D1 H5 T6  
  
Z:\>mount c d:\assembly  
Drive C is mounted as local directory d:\assembly\  
  
Z:\>c:  
  
C:\>debug  
-e ds:000 23 01 25 00  
-e ds:006 2a 2a 2a  
-e cs:100 a1 00 00 03 06 02 00  
-e cs:107 a3 04 00 cb  
-RS
```

(2) 用 D 命令检查数据段、代码段内容。

—D DS: 0

—D CS: 100

```
-d ds:0
073F:0000 23 01 25 00 00 EA 2A 2A-2A DE 4F 03 A3 01 8A 03
073F:0010 A3 01 17 03 A3 01 92 01-01 01 01 00 02 FF FF FF
073F:0020 FF FF FF FF FF FF FF FF-FF FF FF FF 00 00 00
073F:0030 00 00 14 00 18 00 3F 07-FF FF FF FF 00 00 00 00
073F:0040 05 00 00 00 00 00 00 00-00 00 00 00 00 00 00
073F:0050 CD 21 CB 00 00 00 00 00-00 00 00 00 00 20 20 20
073F:0060 20 20 20 20 20 20 20 20-00 00 00 00 00 20 20
073F:0070 20 20 20 20 20 20 20 20-00 00 00 00 00 00 00
-d cs:100
073F:0100 A1 00 00 03 06 02 00 A3-04 00 CB 00 00 00 00 00
073F:0110 00 00 00 00 00 00 00 00-00 00 00 00 34 00 2E 07
073F:0120 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00
073F:0130 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00
073F:0140 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00
073F:0150 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00
073F:0160 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00
073F:0170 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00
-S
```

(3) 用 T 命令逐条执行上述程序，注意观察每条指令执行后相应寄存器的变化。

```
-t=100
AX=0123 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=0103  NU UP EI PL NZ NA PO NC
073F:0103 03060200 ADD AX,[0002] DS:0002=0025
-t
AX=0148 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=0107  NU UP EI PL NZ NA PE NC
073F:0107 A30400 MOV [0004],AX DS:0004=EA00
-t
^ Error
-t
AX=0148 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=010A  NU UP EI PL NZ NA PE NC
073F:010A CB RETF
-S
```

(4) 用 U 命令反汇编本程序。

—U 100 10A

```
-u 100 10a
073F:0100 A10000 MOV AX,[0000]
073F:0103 03060200 ADD AX,[0002]
073F:0107 A30400 MOV [0004],AX
073F:010A CB RETF
-S
```

(5) 用 R 命令修改 IP 的内容为 0100 重新执行上述程序。

—R IP

显示: IP 010A

```
-r ip
IP 010A
:S
```

(6) 用 G 命令运行程序。

—G =CS: 0100 107

观察: IP= AX=

—G =CS: 0100 10A

观察: IP= AX=

```
-g =cs:0100 107
AX=0148 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=0107  NU UP EI PL NZ NA PE NC
073F:0107 A30400      MOV     [0004],AX      DS:0004=0148
-g =cs:0100 10a
AX=0148 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=010A  NU UP EI PL NZ NA PE NC
073F:010A CB          RETF
-S
```

3. 自己试着写段程序:

```
-e 100 '1234567890'
-a 10a
073F:010A xxx:010A cld
^ Error
073F:010A cld
073F:010B mov si,100
073F:010E mov di,200
073F:0111 mov cx,a
073F:0114 rep movsb
073F:0116
-g =10a 116
AX=0148 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=010A DI=020A
DS=073F ES=073F SS=073F CS=073F IP=0116  NU UP EI PL NZ NA PE NC
073F:0116 0000      ADD     [BX+SI],AL      DS:010A=FC
--d 100 l a
^ Error
--d 100 l a
073F:0100 31 32 33 34 35 36 37 38-39 30      1234567890
--d es:200 l a
073F:0200 31 32 33 34 35 36 37 38-39 30      1234567890
-S
```

四、实验总结

本次实验熟练掌握了 Debug 环境及常用命令（A、D、E、T、G 等）。首先通过 E 命令输入机器码完成了内存两数求和，利用 T 命令单步追踪验证了运算结果（0148H）的正确性。随后使用 A 命令编写汇编程序，利用 REP MOVSB 指令成功实现了字符串的内存复制。通过实验，深入理解了寄存器状态变化及指令执行的底层逻辑。

实验二、熟悉指令系统

一、实验目的:

- 1.熟悉 8088/8086 的指令书写格式、功能。
- 2.能熟练运用数据传递、算术、逻辑三类指令。

二、实验任务及要求：

- 1.用 DEBUG 调试程序功能编程序计算 $(W - (X \times Y + Z - 540)) / X$ 。其中 X、Y、Z、W 均为 16 位带符号数，要求上式计算结果的商存入 AX 寄存器中，余数存入 DX 寄存器。掌握算术指令的用法。
- 2.用 DEBUG 编程并查看逻辑指令的运行结果，掌握逻辑指令的用法。
- 3.熟悉其它几组指令的功能和用法，并区分字、字节操作。

三、实验过程及结果：

- 1.预先编制程序片段，实现 $(W - (X \times Y + Z - 540)) / X$ ，将计算结果的商存入 AX 寄存器中，余数存入 DX 寄存器。
- 2.可以在 Debug 程序中使用 E 命令为变量 X、Y、Z、W 设置初值。
- 3.使用 A 命令汇编程序片段，并用 G 或 T 命令执行，用 R 或 D 命令查看结果。
- 4.使用 A 命令汇编几条逻辑指令，并运行，查看结果。

```

C:\>debug
-e ds:0 00 12 34 00 45 f0 4a 03
-a 100
073F:0100 mov ax, [01]
073F:0103 imul word ptr [2]
073F:0107 add ax, [4]
073F:010B adc dx, 0
073F:010E sub ax, 21c
073F:0111 sbb dx, 0
073F:0114 mov cx, ax
073F:0116 mov bx, dx
073F:0118 mov ax, [6]
073F:011B cwd
073F:011C sub ax, cx
073F:011E sbb dx,bx
073F:0120 idiv word ptr[0]
073F:0124 g =100 124
Error
073F:0124
-g =100 124

AX=FFBF BX=0004 CX=9629 DX=FF21 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=0124 NU UP EI NG NZ AC PO CY
073F:0124 0000 ADD [BX+SI],AL DS:0004=45
-S

```

```

AX=1234 BX=0004 CX=9629 DX=FF21 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=0203 NU UP EI NG NZ AC PO CY
073F:0203 25FF00 AND AX,00FF
-t
AX=0034 BX=0004 CX=9629 DX=FF21 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=0206 NU UP EI PL NZ NA PO NC
073F:0206 0D00F0 OR AX,F000
-t
AX=F034 BX=0004 CX=9629 DX=FF21 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=0209 NU UP EI NG NZ NA PO NC
073F:0209 31C0 XOR AX,AX
-t
AX=0000 BX=0004 CX=9629 DX=FF21 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=020B NU UP EI PL ZR NA PE NC
073F:020B F7D0 NOT AX
-t
AX=FFFF BX=0004 CX=9629 DX=FF21 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=020D NU UP EI PL ZR NA PE NC
073F:020D 0000 ADD [BX+SI],AL DS:0004=45
-S

```

```

073F:0300 mov word ptr [0], 1111
073F:0306 mov byte ptr[0],aa
073F:030B r ip
^ Error
073F:030B 300
^ Error
073F:030B
-r ip
IP 020D
:300
-t
AX=FFFF BX=0004 CX=9629 DX=FF21 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=0306 NU UP EI PL ZR NA PE NC
073F:0306 C6060000AA MOV BYTE PTR [0000],AA DS:0000=11
-d ds:0
073F:0000 11 11 34 00 45 F0 4A 03-AD DE 4F 03 A3 01 8A 03 ..4.E.J...0....
073F:0010 A3 01 17 03 A3 01 92 01-01 01 01 00 02 FF FF FF .....
073F:0020 FF FF FF FF FF FF FF-FF FF FF FF 00 00 00 00 .....
073F:0030 00 00 14 00 18 00 3F 07-FF FF FF FF 00 00 00 00 .....?.....
073F:0040 05 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
073F:0050 CD 21 CB 00 00 00 00 00-00 00 00 00 20 20 20 20 !?.....
073F:0060 20 20 20 20 20 20 20 20-00 00 00 00 00 20 20 20 .....
073F:0070 20 20 20 20 20 20 20 20-00 00 00 00 00 00 00 00 .....
-S

```

四、实验总结

本次实验完成了 8086 指令系统的综合练习。首先利用 Debug 编写程序计算了带符号公式 $\$(W-(X \times Y+Z-540))/X\$,$ 熟练运用了 IMUL/IDIV、CWD 符号扩展及 ADC/SBB 多字节运算指令，成功获取了商与余数。其次，通过调试 AND/OR/XOR 等逻辑指令验证了位操作功能。最后，通过对比内存变化，清晰区分了数据传送中字（Word）与字节（Byte）操作的用法。通过上机实践，深刻理解了 8086 指令格式及功能。特别是掌握了带符号数运算中符号扩展的必要性，以及利用 Debug 观察寄存器和内存变化来验证程序逻辑的方法。明确了操作数长度对运行结果的决定性影响，有效提升了汇编语言的编程与调试能力。

实验 3、建立运行汇编语言程序

一、实验目的：

- 1.掌握汇编语言程序格式及其它命令的作用。

2.掌握汇编、连接、运行汇编程序的全过程，并能检查修改程序简单错误。

二、实验内容：

1.编写程序，比较两个字符串 STRING1 和 STRING2 所含的字符是否相同。若相同，则显示'Match'；否则，显示'No match'。

2.编辑、汇编、连接这个汇编语言源程序，形成.EXE 文件。

3.通过 DEBUG 调试运行该可执行文件，查看运行结果。

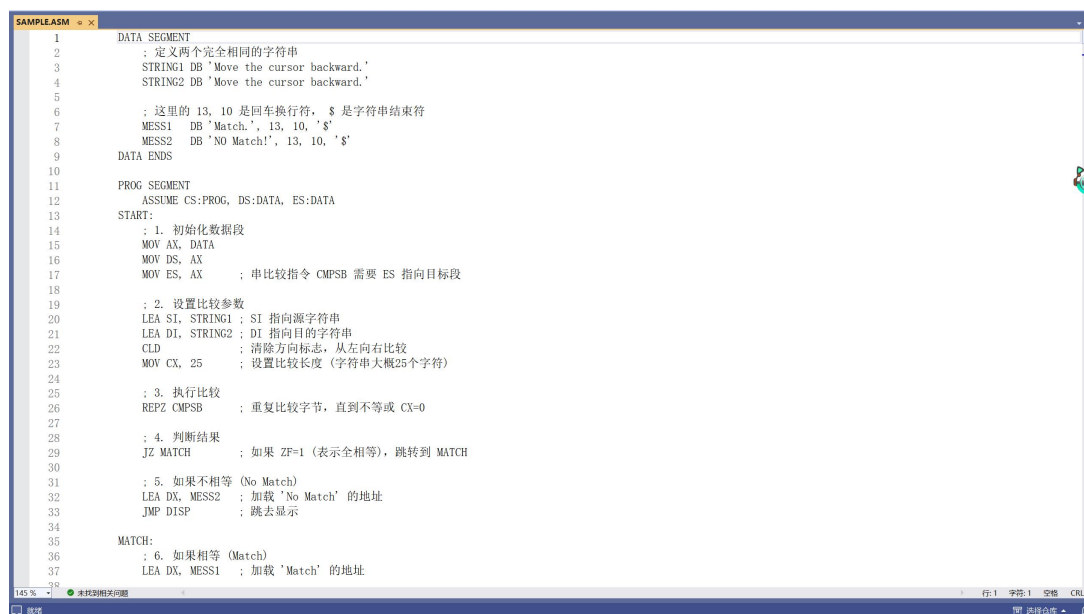
三、实验过程及结果：

1.根据要求编制汇编源程序。

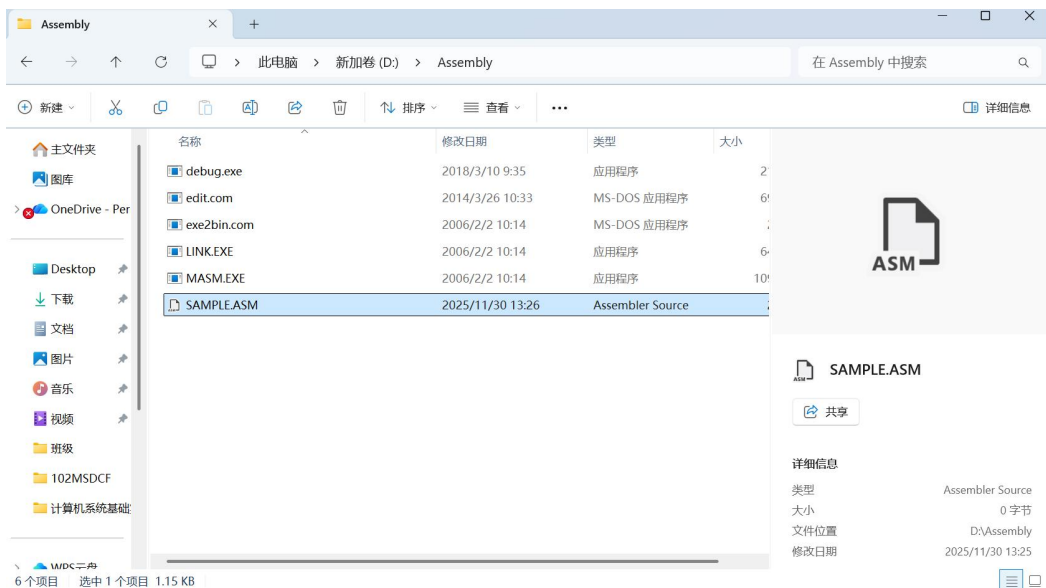
2.使用记事本或其它字处理软件(Edit.exe)编辑该源文件。

3.使用 Masm、Link 程序汇编、连接该文件，形成可执行的.EXE 文件。

4.也可以使用 **Debug 调试** 运行这个.EXE 文件，并查看运行结果。对实验内容修改两个字符串的内容，使它们互不相同，再观察程序的运行结果。



```
1  DATA SEGMENT
2      ; 定义两个完全相同的字符串
3      STRING1 DB 'Move the cursor backward.'
4      STRING2 DB 'Move the cursor backward.'
5
6      ; 这里的 13, 10 是回车换行符, $ 是字符串结束符
7      MESS1 DB 'Match.', 13, 10, '$'
8      MESS2 DB 'NO Match!', 13, 10, '$'
9  DATA ENDS
10
11  PROG SEGMENT
12      ASSUME CS:PROG, DS:DATA, ES:DATA
13  START:
14      ; 1. 初始化数据段
15      MOV AX, DATA
16      MOV DS, AX
17      MOV ES, AX      ; 串比较指令 CMPSB 需要 ES 指向目标段
18
19      ; 2. 设置比较参数
20      LEA SI, STRING1 ; SI 指向源字符串
21      LEA DI, STRING2 ; DI 指向目的字符串
22      CLD             ; 清除方向标志, 从左向右比较
23      MOV CX, 25      ; 设置比较长度 (字符串大概25个字符)
24
25      ; 3. 执行比较
26      REPZ CMPSB      ; 重复比较字节, 直到不等或 CX=0
27
28      ; 4. 判断结果
29      JZ MATCH        ; 如果 ZF=1 (表示全相等), 跳转到 MATCH
30
31      ; 5. 如果不相等 (No Match)
32      LEA DX, MESS2    ; 加载 'No Match' 的地址
33      JMP DISP        ; 跳去显示
34
35  MATCH:
36      ; 6. 如果相等 (Match)
37      LEA DX, MESS1    ; 加载 'Match' 的地址
```

```

DOS
BOX
DOSBox 0.74, Cpu speed: 3000 cycles, Frames...
Drive C is mounted as local directory d:\assembly\
Z:\>c:
C:\>masm sample.asm;
Microsoft (R) Macro Assembler Version 5.10
Copyright (C) Microsoft Corp 1981, 1988. All rights reserved.

50110 + 461247 Bytes symbol space free

0 Warning Errors
0 Severe Errors

C:\>link sample.obj;
Microsoft (R) Overlay Linker Version 3.64
Copyright (C) Microsoft Corp 1983-1988. All rights reserved.

LINK : warning L4021: no stack segment

C:\>sample
Match.

C:\>S

```

```

0076F:000F FC      CLD
0076F:0010 B91900    MOV     CX,0019
0076F:0013 F3      REPZ
0076F:0014 A6      CMPSB
0076F:0015 7407     JZ      001E
0076F:0017 8D163B00    LEA     DX,[003B]
0076F:001B EB05     JMP     0022
0076F:001D 90      NOP
0076F:001E 8D163200    LEA     DX,[0032]
-t
AX=076A BX=0000 CX=007A DX=0000 SP=0000 BP=0000 SI=0000 DI=0000
DS=075A ES=075A SS=0769 CS=076F IP=0003  NU UP EI PL NZ NA PO NC
0076F:0003 8ED8      MOV     DS,AX
-t
AX=076A BX=0000 CX=007A DX=0000 SP=0000 BP=0000 SI=0000 DI=0000
DS=076A ES=076A SS=0769 CS=076F IP=0005  NU UP EI PL NZ NA PO NC
0076F:0005 8EC0      MOV     ES,AX
-t
AX=076A BX=0000 CX=007A DX=0000 SP=0000 BP=0000 SI=0000 DI=0000
DS=076A ES=076A SS=0769 CS=076F IP=0007  NU UP EI PL NZ NA PO NC
0076F:0007 8D360000    LEA     SI,[0000]          DS:0000=6F4D
-S

```



```
DOSBox 0.74, Cpu speed: 3000 cycles, Frames...
-t
AX=076A BX=0000 CX=007A DX=0000 SP=0000 BP=0000 SI=0000 DI=0000
DS=075A ES=075A SS=0769 CS=076F IP=0003 NU UP EI PL NZ NA PO NC
076F:0003 8ED8 MOV DS,AX
-t
AX=076A BX=0000 CX=007A DX=0000 SP=0000 BP=0000 SI=0000 DI=0000
DS=076A ES=075A SS=0769 CS=076F IP=0005 NU UP EI PL NZ NA PO NC
076F:0005 8EC0 MOV ES,AX
-t
AX=076A BX=0000 CX=007A DX=0000 SP=0000 BP=0000 SI=0000 DI=0000
DS=076A ES=076A SS=0769 CS=076F IP=0007 NU UP EI PL NZ NA PO NC
076F:0007 8D360000 LEA SI,[0000] DS:0000=6F4D
-d ds:0
076A:0000 4D 6F 76 65 20 74 68 65-20 63 75 72 73 6F 72 20 Move the cursor
076A:0010 62 61 63 6B 77 61 72 64-2E 4D 6F 76 65 20 74 68 backward.Move th
076A:0020 65 20 63 75 72 73 6F 72-20 62 61 63 6B 77 61 72 e cursor backwar
076A:0030 64 2E 4D 61 74 63 68 2E-0D 0A 24 4E 4F 20 4D 61 d.Match...$NO Ma
076A:0040 74 63 68 21 0D 0A 24 00-00 00 00 00 00 00 00 00 tch!..$.
076A:0050 BB 6A 07 8E D8 8E C0 8D-36 00 00 8D 3E 19 00 FC .j.....6...>...
076A:0060 B9 19 00 F3 A6 74 07 8D-16 3B 00 EB 05 90 8D 16 .....t...:.....
076A:0070 32 00 B4 09 CD 21 B4 4C-CD 21 C5 FF 76 FC EB 1B 2....!.L.!..v...
-Ss
```

```
DOSBox 0.74, Cpu speed: 3000 cycles, Frames...
AX=076A BX=0000 CX=007A DX=0000 SP=0000 BP=0000 SI=0000 DI=0000
DS=076A ES=075A SS=0769 CS=076F IP=0005 NU UP EI PL NZ NA PO NC
076F:0005 8EC0 MOV ES,AX
-t
AX=076A BX=0000 CX=007A DX=0000 SP=0000 BP=0000 SI=0000 DI=0000
DS=076A ES=076A SS=0769 CS=076F IP=0007 NU UP EI PL NZ NA PO NC
076F:0007 8D360000 LEA SI,[0000] DS:0000=6F4D
-d ds:0
076A:0000 4D 6F 76 65 20 74 68 65-20 63 75 72 73 6F 72 20 Move the cursor
076A:0010 62 61 63 6B 77 61 72 64-2E 4D 6F 76 65 20 74 68 backward.Move th
076A:0020 65 20 63 75 72 73 6F 72-20 62 61 63 6B 77 61 72 e cursor backwar
076A:0030 64 2E 4D 61 74 63 68 2E-0D 0A 24 4E 4F 20 4D 61 d.Match...$NO Ma
076A:0040 74 63 68 21 0D 0A 24 00-00 00 00 00 00 00 00 00 tch!..$.
076A:0050 BB 6A 07 8E D8 8E C0 8D-36 00 00 8D 3E 19 00 FC .j.....6...>...
076A:0060 B9 19 00 F3 A6 74 07 8D-16 3B 00 EB 05 90 8D 16 .....t...:.....
076A:0070 32 00 B4 09 CD 21 B4 4C-CD 21 C5 FF 76 FC EB 1B 2....!.L.!..v...
-e ds:0 'X'
^ Error
-e ds:0 'X'
-g
NO Match!
Program terminated normally
-S
```

四、实验总结

本次实验掌握了汇编程序开发的完整流程（编辑-汇编-连接-运行）。首先编写了字符串比较源程序（SAMPLE.ASM），使用 MASM 汇编生成目标文件（.OBJ），再用 LINK 连接生成可执行文件（.EXE）。直接运行程序输出了“Match.”。随后利用 Debug 加载 EXE 文件，在初始化 DS/ES 段寄存器后，使用 E 命令修改内存中源字符串的首字符，通过 G 命令运行，成功输出“NO Match!”，验证了程序在不同数据下的逻辑正确性。通过实验，熟练掌握了 MASM 和 LINK 工具的使用。在编程层面，深刻理解了串操作指令（CMPSB）的工作机制，即必须正确初始化 ES 寄存器（使 ES:DI 指向目的串，DS:SI 指向源串）。同时，学会了在 Debug 中

“动态修改内存数据”来验证程序不同分支（JZ/JNZ）的调试技巧，这比反复修改源码重新编译更加高效。

实验 4、循环程序设计

一、实验目的：

- 1.加深对循环程序的理解。
- 2.能构造出正确的循环结构并能实现较复杂的算法。

二、实验内容：

1.编制程序，要求如下：从键盘接收一个四位的 16 进制数，在终端上显示与它等值的二进制数。

三、实验过程及结果：

1. 根据要求，画出程序框图，写出汇编源代码。
2. 上机编辑源程序，并汇编、连接、调试运行，查看结果。

编译源代码如下：

```
01 CODE SEGMENT
02     ASSUME CS:CODE
03 Begin:
04     MOV BX, 0      ; BX 用来存放最终结果（累加器）
05     MOV CH, 4      ; 循环计数器：需要输入 4 个 16 进制位
06     MOV CL, 4      ; 移位次数：每次左移 4 位
07
08 INPUT:
09     SHL BX, CL      ; 将之前输入的数左移 4 位，腾出低 4 位给新输入的数
10
11     ; --- 从键盘读取一个字符 ---
12     MOV AH, 1
13     INT 21H
14
15     ; --- 判断字符有效性及转换 ---
16     CMP AL, 30H     ; 与 '0' 比较
17     JB INPUT        ; 小于 '0', 非法，重新输入
18
19     CMP AL, 39H     ; 与 '9' 比较
20     JA AF           ; 大于 '9', 可能是 A-F, 跳转到 AF 处理
21
```

```

22      ; 如果是 0-9
23      AND AL, 0FH      ; 去掉高 4 位, 把 ASCII '0'(30H) 变成数值 0
24      JMP BINARY      ; 跳去合并
25
26 AF:
27      AND AL, 11011111B ; (即 AND AL, 0DFH) 将小写字母转为大写 (a->A)
28
29      CMP AL, 'A'      ; 是否小于 'A' ?
30      JB INPUT        ; 非法字符 (比如在 9 和 A 之间的符号), 重新输入
31
32      CMP AL, 'F'      ; 是否大于 'F' ?
33      JA INPUT        ; 非法字符, 重新输入
34
35      SUB AL, 37H      ; 核心转换: 'A'(41H) - 37H = 0AH (10)
36
37 BINARY:
38      OR BL, AL        ; 将转换后的 4bit 值合并到 BX 的低 4 位
39      DEC CH          ; 计数器减 1
40      JNZ INPUT        ; 如果还没输完 4 位, 继续循环
41
42      ; --- (可选) 输出一个回车换行, 让结果显示在下一行 ---
43      MOV AH, 2
44      MOV DL, 0DH      ; 回车
45      INT 21H
46      MOV DL, 0AH      ; 换行
47      INT 21H
48
49      ; --- 准备显示二进制 ---
50      MOV CX, 16       ; 16 位二进制, 循环 16 次
51
52 DISP:
53      MOV DL, 0        ; 清零 DL
54      ROL BX, 1        ; BX 循环左移 1 位, 最高位移入 CF 标志位
55      RCL DL, 1        ; 带进位循环左移, 把 CF 移入 DL 的最低位
56                      ; 此时 DL 要么是 0, 要么是 1
57      OR DL, 30H       ; 把数值 0/1 变成字符 '0'/'1'
58
59      MOV AH, 2        ; 显示字符
60      INT 21H
61      LOOP DISP       ; CX 减 1, 不为 0 则继续循环
62
63 STOP:
64      MOV AH, 4CH
65      INT 21H

```

```
66 CODE ENDS
67 END Begin
```

```
TEST4.ASM*
14 MOV BX, 0      ; BX用来存放最终结果(累加器)
15 MOV CH, 4      ; 循环计数器: 需要输入4个16进制位
16 MOV CL, 4      ; 移位次数: 每次左移4位
17
18 INPUT:
19 SHL BX, CL      ; 将之前输入的数左移4位, 腾出低4位给新输入的数
20
21 ; --- 从键盘读取一个字符 ---
22 MOV AH, 1
23 INT 21H
24
25 ; --- 判断字符有效性及转换 ---
26 CMP AL, 30H     ; 与 '0' 比较
27 JB INPUT        ; 小于 '0', 非法, 重新输入
28
29 CMP AL, 39H     ; 与 '9' 比较
30 JA AF          ; 大于 '9', 可能是 A-F, 跳转到 AF 处理
31
32 ; 如果是 0-9
33 AND AL, 0FH     ; 去掉高4位, 把 ASCII '0' (30H) 变成数值 0
34 JMP BINARY     ; 跳去合并
35
36 AF:
37 AND AL, 11011111B ; (即 AND AL, 0DFH) 将小写字母转为大写 (a->A)
38
39 CMP AL, 'A'     ; 是否小于 'A' ?
40 JB INPUT        ; 非法字符 (比如在 9 和 A 之间的符号), 重新输入
41
42 CMP AL, 'F'     ; 是否大于 'F' ?
43 JA INPUT        ; 非法字符, 重新输入
44
45 SUB AL, 37H     ; 核心转换: 'A' (41H) - 37H = 0AH (10)
46
47 BINARY:
48 OR BL, AL       ; 将转换后的 4bit 值合并到 BX 的低 4 位
49 DEC CH         ; 计数器减 1
50 JNZ INPUT       ; 如果还没读完4位, 继续循环
51
52 ; --- (可选) 输出一个回车换行, 让结果显示在下一行 ---
53 MOV AH, 2
54 MOV DL, 0DH    ; 回车
55 INT 21H
56 MOV DL, 0AH    ; 换行
57 INT 21H
58
59 ; --- 准备显示二进制 ---
60 MOV CX, 16     ; 16位二进制, 循环16次
61
62 DISP:
63 MOV DL, 0      ; 清零 DL
64 ROL BX, 1      ; BX 循环左移 1 位, 最高位移入 CF 标志位
65 JNC DISP       ; 如果没有进位, 继续循环
66 MOV DL, 48H    ; 显示 '0'
67 INT 21H
68 JNC DISP       ; 如果没有进位, 继续循环
69 MOV DL, 22H    ; 显示 '1'
70 INT 21H
71 JMP DISP
72
73 END
```

```
Microsoft (R) Macro Assembler Version 5.10
Copyright (C) Microsoft Corp 1981, 1988. All rights reserved.

50124 + 461233 Bytes symbol space free

0 Warning Errors
0 Severe Errors

C:\>link test4.obj;

Microsoft (R) Overlay Linker Version 3.64
Copyright (C) Microsoft Corp 1983-1988. All rights reserved.

LINK : warning L4021: no stack segment

C:\>test4
1234
0001001000110100
```

四、实验总结

本次实验编写了“十六进制转二进制显示”的程序，重点掌握了循环结构与位操作指令的应用。利用循环接收 4 个字符，通过比较指令（CMP）判断数字（0 - 9）与字母（A - F），并利用 SUB 指令将 ASCII 码转换为对应数值。特别处理了大小写转换（AND AL, 0DFH）。使

用 SHL 左移指令将输入的 4 位数据依次拼合到 BX 寄存器中。通过 ROL 循环左移指令配合 RCL 带进位循环左移，将 BX 中的每一位移入 DL，再转换为 ASCII 码（'0' 或 '1'）调用 DOS 中断显示。通过本次实验，深刻理解了移位指令在进制转换中的核心作用，以及如何利用标志位（CF）在寄存器间传递数据位。